Intelligent Document Processing Platform

Functional Document (FD)

GITHUB LINK: https://github.com/juweriya1/ai-document-processing

BY

Team Lead: ***Fatima Naeem 26933***

Member(s): ***Maheen Muhammad Rizwan 27118***

Juweriya Bint Nasir 26787

PROJECTS COMMITTEE (PC)

SUPERVISOR: ***Dr. Rizwan Ahmed Khan***

SUBMITTED TO

PROJECTS Manager – FYP

ON

DATE (18/11/2025)

IBA SMCS

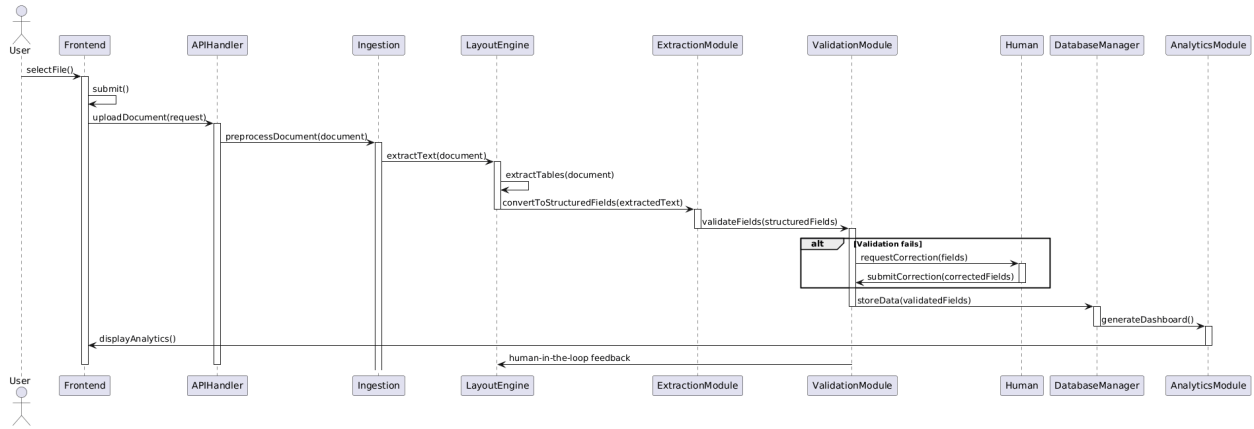
School of Mathematics and Computer Science

Table of Contents

PROJECTS COMMITTEE (PC)..... 1
1. Interaction Diagrams..... 3
2. Class Diagram and Interface Specification..... 4
3. System Architecture and System Design..... 5
    3.1. Architectural Style..... 5
    3.2. Identifying Subsystems..... 5
    3.3. Mapping Subsystems to Hardware..... 6
    3.4. Persistent Data Storage..... 6
    3.5. Network Protocol..... 6
    3.6. Global Control Flow..... 6
    3.7. Hardware/Software Requirements..... 6
4. Algorithms and Data Structures..... 6
5. User Interface Design and Implementation..... 7
6. Design of Tests..... 7

System Design

1. Interaction Diagrams

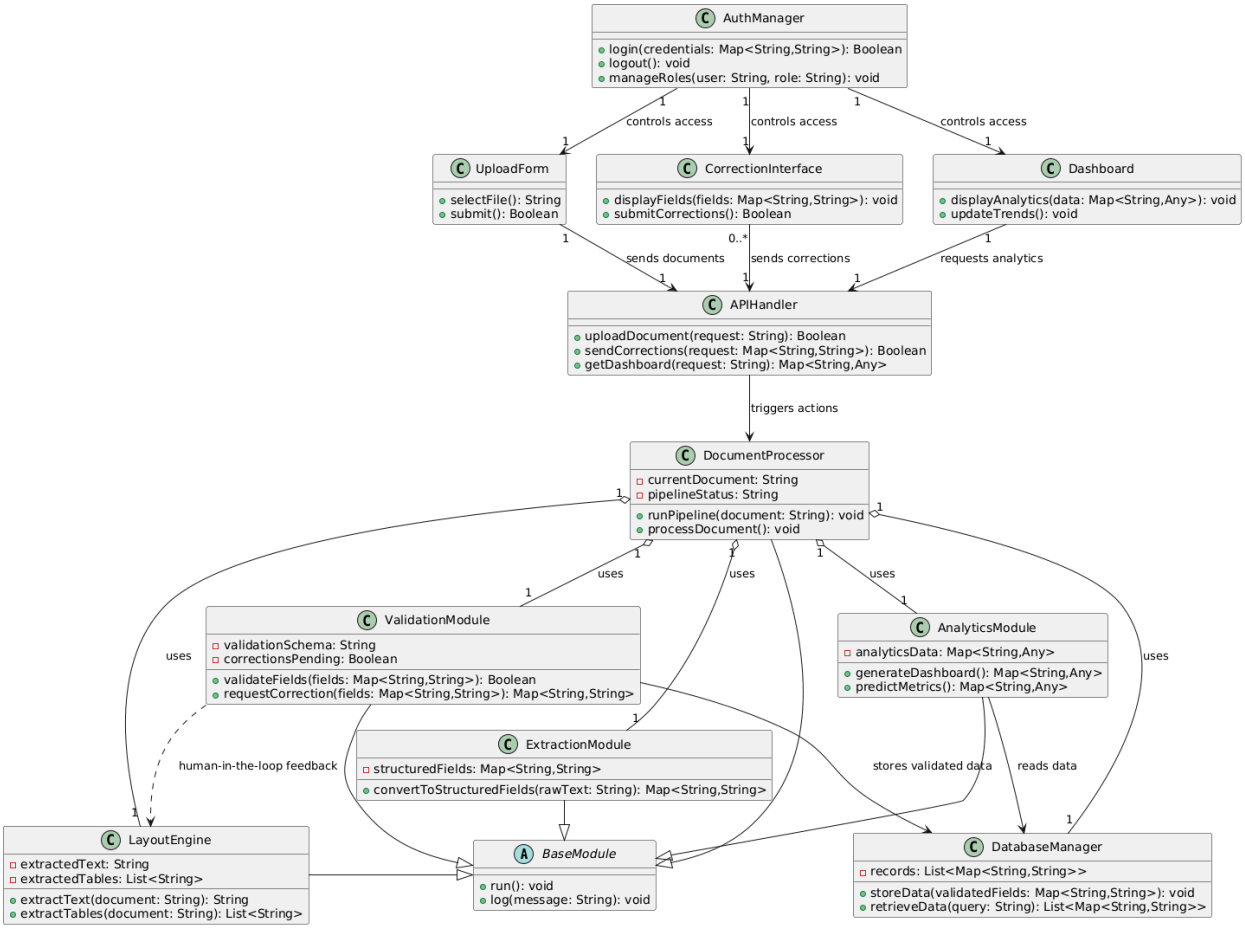


The system works in the following way:

1. The user uploads a document (PDF, image, or Word file) through the frontend.
2. The backend processes the document using a **layout-aware document processing engine** to extract text, tables, and figures.
3. Extracted data is passed to the **information extraction module** which identifies important fields like invoice numbers, dates, line items, or clauses.
4. The extracted fields are checked by a **validation module** against predefined schemas. The user can manually correct errors through the frontend.
   Validated data is stored in the database for long-term use.
5. Analytics and dashboards are updated to show summaries, trends, and predictive insights.

Data flows mainly from frontend → backend → database → dashboard. The human-in-the-loop feedback can interrupt the flow to correct extraction results.

2. Class Diagram and Interface Specification



Backend Classes:

- DocumentProcessor: Main controller that runs the pipeline for each document.
- LayoutEngine: Core module that extracts text and tables from documents.
- ExtractionModule: Converts raw text into structured data fields.
- ValidationModule: Checks extracted fields against schemas; manages corrections.
- DatabaseManager: Stores and retrieves validated data.
- AnalyticsModule: Generates dashboards and predictive metrics.
- APIHandler: Provides routes for the frontend.

Frontend Components:

- UploadForm: Lets the user upload documents.
- CorrectionInterface: Allows manual verification and editing of extracted fields.
- Dashboard: Shows analytics, trends, and predictions.
- AuthManager: Manages login and user roles.

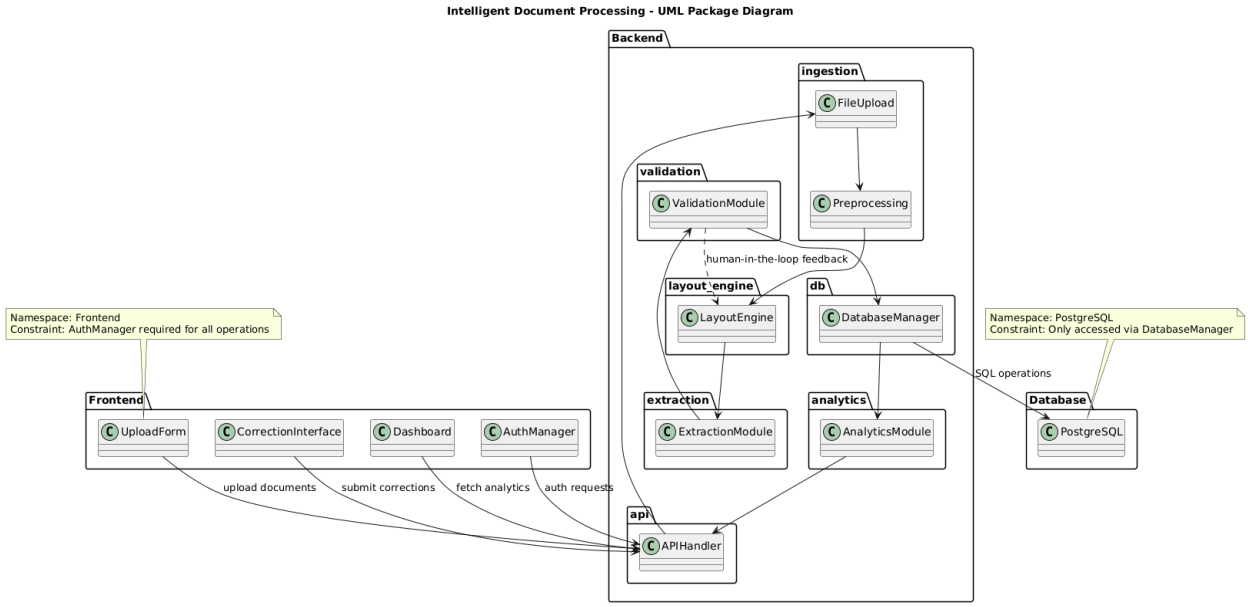
All backend modules communicate through function calls and APIs. The layout engine is designed so it can be improved and fine-tuned as the project progresses.

3. System Architecture and System Design

3.1. Architectural Style

The system follows a **monolithic but modular design**. All backend functionality is in a single application but divided into modules for ingestion, layout processing, extraction, validation, database, and analytics. Frontend is built with React and communicates with backend via REST APIs.

3.2. Identifying Subsystems



1. Document ingestion (upload + preprocessing)
2. Layout-aware extraction module (text + tables + figures)
3. Information extraction (entity identification)
4. Validation and human-in-the-loop feedback
5. Database storage
6. Analytics and dashboards
7. Frontend interface
8. Authentication and authorization

3.3. Mapping Subsystems to Hardware

- Backend and database run on a single machine.
- Frontend runs on user machines.
- GPU may be used for layout processing.

3.4. Persistent Data Storage

- Database: PostgreSQL for structured data.
- Tables include:
    - Document metadata (document ID, filename, status)
    - Extracted fields (field name, value, confidence score, validation status)
    - User corrections
    - Analytics summaries

3.5. Network Protocol

- REST APIs over HTTPS
- JSON payloads
- JWT authentication for secure endpoints

3.6. Global Control Flow

- Event-driven, mostly linear: Upload → layout processing → extraction → validation → database → analytics → dashboard
- Human corrections can interrupt flow and trigger updates in extraction results

3.7. Hardware/Software Requirements

- OS: Linux/Windows/MacOS
- RAM: 8+ GB
- GPU: For layout processing
- Python 3.11+, FastAPI, React.js
- PostgreSQL 14+
- Libraries: PyTorch/TensorFlow, NLP libraries, document processing modules

4. Algorithms and Data Structures

Algorithms:

- Layout-aware document processing for text and table extraction
- Entity extraction using ML + rule-based methods
- Schema-based validation with human corrections
- Analytics and predictive models for trends and anomaly detection

Data Structures:

- Python lists and dictionaries for intermediate data
- Hash tables for fast validation lookup
- Relational database tables for final structured storage

5. User Interface Design and Implementation

- The UI design now accommodates visual review of layout-aware extraction.
- Conceptual design supports highlighting extracted sections
- Design prioritizes clarity of steps and minimal user cognitive load

6. Design of Tests

Unit Tests:

- Document upload validation
- Layout extraction accuracy
- Entity extraction correctness
- Schema validation
- Database storage and retrieval
- API endpoint tests

Integration Tests:

- Full pipeline: Upload → layout extraction → extraction → validation → database → analytics
- Human-in-the-loop scenarios tested

Research/Improvement Tests:

1. Field-level Accuracy: Precision, Recall, F1 for structured fields (tables, forms, invoices).
2. Character / Word Accuracy: CER (Character Error Rate) or word-level error rate.
3. Layout & Structure Retention: Correct reconstruction of tables, headings, sections.
4. Exact / Partial Match Rate: % of documents with all or most fields correctly extracted.
7. Human-in-the-loop Metrics: Correction rate, validation time, confidence vs error correlation.
8. Comparative Improvement: % improvement over baseline models; error type analysis.